JavaScript is disabled on your browser.

compact1, compact2, compact3

java.io

Class LineNumberInputStream

java.lang.Object
    java.io.InputStream
        java.io.FilterInputStream
            java.io.LineNumberInputStream

All Implemented Interfaces:

`Closeable`, `AutoCloseable`

Deprecated.

This class incorrectly assumes that bytes adequately represent characters. As of JDK 1.1, the preferred way to operate on character streams is via the new character-stream classes, which include a class for counting line numbers.

```
@Deprecated
public class LineNumberInputStream
extends FilterInputStream
```

This class is an input stream filter that provides the added functionality of keeping track of the current line number.

A line is a sequence of bytes ending with a carriage return character (`'\r'`), a newline character (`'\n'`), or a carriage return character followed immediately by a linefeed character. In all three cases, the line terminating character(s) are returned as a single newline character.

The line number begins at `0`, and is incremented by `1` when a `read` returns a newline character.

Since:

`JDK1.0`

See Also:

`LineNumberReader`

Field Summary

Fields inherited from class java.io.FilterInputStream

in

Constructor Summary

Constructors

Constructor	Description
`LineNumberInputStream(InputStream in)`	**Deprecated.** Constructs a newline number input stream that reads its input from the specified input stream.

Method Summary

Concrete Methods | **Deprecated Methods**

Modifier and Type	Method	Description
int	`available()`	**Deprecated.** Returns the number of bytes that can be read from this input stream without blocking.
int	`getLineNumber()`	**Deprecated.** Returns the current line number.
void	`mark(int readlimit)`	**Deprecated.** Marks the current position in this input stream.
int	`read()`	**Deprecated.**

		Reads the next byte of data from this input stream.
int	**read**(byte[] b, int off, int len)	**Deprecated.** Reads up to `len` bytes of data from this input stream into an array of bytes.
void	**reset**()	**Deprecated.** Repositions this stream to the position at the time the `mark` method was last called on this input stream.
void	**setLineNumber**(int lineNumber)	**Deprecated.** Sets the line number to the specified argument.
long	**skip**(long n)	**Deprecated.** Skips over and discards `n` bytes of data from this input stream.

Methods inherited from class java.io.FilterInputStream

close, markSupported, read

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

LineNumberInputStream

```
public LineNumberInputStream(InputStream in)
```

Deprecated.

Constructs a newline number input stream that reads its input from the specified input stream.

Parameters:

`in - the underlying input stream.`

Method Detail

read

```
public int read()
         throws IOException
```

Deprecated.

Reads the next byte of data from this input stream. The value byte is returned as an `int` in the range `0` to `255`. If no byte is available because the end of the stream has been reached, the value `-1` is returned. This method blocks until input data is available, the end of the stream is detected, or an exception is thrown.

The `read` method of `LineNumberInputStream` calls the `read` method of the underlying input stream. It checks for carriage returns and newline characters in the input, and modifies the current line number as appropriate. A carriage-return character or a carriage return followed by a newline character are both converted into a single newline character.

Overrides:

`read` in class `FilterInputStream`

Returns:

`the next byte of data, or -1 if the end of this stream is reached.`

Throws:

`IOException - if an I/O error occurs.`

See Also:

`FilterInputStream.in, getLineNumber()`

read

```
public int read(byte[] b,
                int off,
                int len)
         throws IOException
```

Deprecated.

Reads up to `len` bytes of data from this input stream into an array of bytes. This method blocks until some input is available.

The `read` method of `LineNumberInputStream` repeatedly calls the `read` method of zero arguments to fill in the byte array.

Overrides:

`read` in class `FilterInputStream`

Parameters:

`b - the buffer into which the data is read.`

`off - the start offset of the data.`

`len - the maximum number of bytes read.`

Returns:

`the total number of bytes read into the buffer, or -1 if there is no more data because the end of this stream has been reached.`

Throws:

`IOException - if an I/O error occurs.`

See Also:

`read()`

skip

```
public long skip(long n)
          throws IOException
```

Deprecated.

Skips over and discards `n` bytes of data from this input stream. The `skip` method may, for a variety of reasons, end up skipping over some smaller number of bytes, possibly `0`. The actual number of bytes skipped is returned. If `n` is negative, no bytes are skipped.

The `skip` method of `LineNumberInputStream` creates a byte array and then repeatedly reads into it until `n` bytes have been read or the end of the stream has been reached.

Overrides:

`skip` in class `FilterInputStream`

Parameters:

`n - the number of bytes to be skipped.`

Returns:

`the actual number of bytes skipped.`

Throws:

`IOException - if an I/O error occurs.`

See Also:

`FilterInputStream.in`

setLineNumber

```
public void setLineNumber(int lineNumber)
```

Deprecated.

Sets the line number to the specified argument.

Parameters:

`lineNumber - the new line number.`

See Also:

`getLineNumber()`

getLineNumber

```
public int getLineNumber()
```

Deprecated.

Returns the current line number.

Returns:

`the current line number.`

See Also:

`setLineNumber(int)`

available

```
public int available()
              throws IOException
```

Deprecated.

Returns the number of bytes that can be read from this input stream without blocking.

Note that if the underlying input stream is able to supply k input characters without blocking, the `LineNumberInputStream` can guarantee only to provide $k/2$ characters without blocking, because the k characters from the underlying input stream might consist of $k/2$ pairs of `'\r'` and `'\n'`, which are converted to just $k/2$ `'\n'` characters.

Overrides:

`available` in class `FilterInputStream`

Returns:

the number of bytes that can be read from this input stream without blocking.

Throws:

IOException - if an I/O error occurs.

See Also:

FilterInputStream.in

mark

```
public void mark(int readlimit)
```

Deprecated.

Marks the current position in this input stream. A subsequent call to the `reset` method repositions this stream at the last marked position so that subsequent reads re-read the same bytes.

The `mark` method of `LineNumberInputStream` remembers the current line number in a private variable, and then calls the `mark` method of the underlying input stream.

Overrides:

mark in class FilterInputStream

Parameters:

readlimit - the maximum limit of bytes that can be read before the mark position becomes invalid.

See Also:

FilterInputStream.in, reset()

reset

```
public void reset()
           throws IOException
```

Deprecated.

Repositions this stream to the position at the time the `mark` method was last called on this input stream.

The `reset` method of `LineNumberInputStream` resets the line number to be the line number at the time the `mark` method was called, and then calls the `reset` method of the underlying input stream.

Stream marks are intended to be used in situations where you need to read ahead a little to see what's in the stream. Often this is most easily done by invoking some general parser. If the stream is of the type handled by the

parser, it just chugs along happily. If the stream is not of that type, the parser should toss an exception when it fails, which, if it happens within readlimit bytes, allows the outer code to reset the stream and try another parser.

Overrides:

`reset` in class `FilterInputStream`

Throws:

`IOException` - if an I/O error occurs.

See Also:

`FilterInputStream.in`, `mark(int)`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.